SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Grama Jyothi Billing Program

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

TASK 1 – Understand the Problem and Identify the Data

Aim

To identify every input, output, constant and rule of the billing problem and justify each data item.

Analysis

Item	Kind	Justification
Household number	Input	Identifier, positive, up to 800,000 in a district; never used in arithmetic, so it is a label.
Previous / current reading	Input	Non-negative whole numbers in 0–65535 (16-bit register). Never fractional.
Category	Input	Small code (e.g. domestic = 1); a char or enum is enough.
Units consumed	Computed	current − previous, with rollover correction; whole and non-negative.
Energy charge	Computed	Slab-wise; fractions of a rupee arise (Rs 2.25) so store in paise.
Fixed charge	Constant	Rs 50 = 5000 paise; does not change in a run.
Total	Output	fixed + energy, in paise; district totals reach billions of paise.
Slab limits & rates	Constants	100, 200 units; 0, 225, 450 paise. Must live in one place so a revision is applied everywhere.

Constants: slab limits, rates, fixed charge, register maximum (65535). **Negative:** only a naive current − previous after rollover (e.g. 120 − 65400); a valid bill is never negative. **Fractional:** rates in rupees (2.25), solved by using paise. **Very large:** district totals (800,000 × ~Rs 275 = 22 billion paise, beyond a 32-bit int).

Rules a correct bill must satisfy:

- Bill ≥ fixed charge ≥ 0 (never negative).
- Total = fixed charge + energy charge, exactly, to the paisa.
- Bill is non-decreasing as units increase (monotonic).
- Units = (current − previous) mod 65536.
- Sum of all bills = sum of the parts, regardless of order of addition.

Assumptions: one meter register per house; at most one rollover per month; a month never exceeds 5,000 units (so a "current < previous" jump larger than that is treated as a data error, not a rollover); one tariff for all categories; amounts in Indian rupees.

Alternative Concept – readings stored as text ("01530")

Each reading would be kept as char[6]. To subtract, it must be converted digit by digit (value = value*10 + digit), and to compare it can be compared directly with strcmp when zero-padded to the same width, or after conversion.

	Integer (unsigned short)	Text (d digits)
Convert	Once at input: O(d)	O(d) every time it is used
Compare / subtract	O(1)	O(d) (strcmp) or O(d) convert + O(1)
Space per reading	2 bytes	d + 1 bytes (6 bytes for 5 digits)
For 800,000 readings	1.6 MB	4.8 MB

Since d ≤ 5 is tiny, the time difference is small, but integers are 3× smaller, avoid repeated conversion and remove parsing bugs. Text is only preferable for keeping leading zeros for display.

Result: Every data item, constant and rule is identified and justified.

TASK 2 – Choose the Data Types

Aim

To choose a suitable C type for each data item, using sizeof and the limits of the types on the system.

Data-type table (measured on my system)

Data item	Type	sizeof	Range / reason
Meter reading	unsigned short (meter_t)	2	0 – 65535; matches the 16-bit register
Household number	int (or unsigned int)	4	up to 2,147,483,647 ≥ 800,000
Units consumed	unsigned int	4	0 – 4,294,967,295; never negative
Category	char	1	small code
One bill (paise)	long	8*	±9.22×10^18 on Linux/64-bit (*4 bytes on Windows!)
Total for district	long long	8	±9.22×10^18 guaranteed at least 64-bit
Rates, slab limits	int / long (macros)	4 / 8	whole paise / whole units

Measured: char 1, short 2, int 4, long 8, long long 8, float 4, double 8. INT_MAX = 2147483647, LLONG_MAX = 9223372036854775807, USHRT_MAX = 65535, FLT_DIG = 6, DBL_DIG = 15. A district total of about 22,040,000,000 paise exceeds INT_MAX, so long long is the safe choice.

Why not float rupees? Values such as 2.25 are exact in binary, but 162.60 or 0.10 are not; float has only about 6–7 significant digits, so errors accumulate. Storing paise in an integer is exact because every amount is a whole number of paise.

Rollover: after 65535 an unsigned short wraps to 0 (arithmetic is modulo 65536).

Precision demonstration

```
float f=0; long p=0;
for (int i=0;i<4000;i++){ f+=162.60f; p+=16260; }
/* float  = 650405.1875   (wrong by Rs 5.19)
   paise  = Rs 650400.00  (exact)          */
```

Complete the Code – TODO 1

```
#include <stdio.h>
#include <limits.h>
typedef unsigned short meter_t;
unsigned int consumption(meter_t previous, meter_t current)
{
   return (meter_t)(current - previous);
}
int main(void)
{
   printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n", sizeof(meter_t), (unsigned)USHRT_MAX);
   printf("Consumption: %u units\n", consumption(1200, 1530));
   printf("Consumption: %u units\n", consumption(65400, 120));
   return 0;
}
```

Output:

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```



Why 256: 120 − 65400 is promoted to int and equals −65280, but converting it to meter_t (unsigned short) reduces it modulo 65536: −65280 + 65536 = 256. This is exactly the true consumption (65400→65535 is 135 units, 65535→0 is 1 unit, 0→120 is 120 units; 135 + 1 + 120 = 256). No if–else is needed.

Alternative Concept – money as double rupees

Bill = 50.0 + slab charges computed with 2.25 and 4.50, accumulated in a double and rounded with round(x*100)/100 once per bill (and again for totals). Time per addition is O(1) in both; space is 8 bytes for double versus 8 bytes for long – the same on a 64-bit system. The difference is error: integer paise has zero error for any n, whereas double has error that grows roughly like n × 2^-53 × total in the worst case (about √n in the typical case). Double is accurate enough to display after rounding, but equality checks and audits can fail (0.1 + 0.2 ≠ 0.3).

Result: Types justified; precision problem shown (Rs 5.19 error with float) and removed by integer paise.

TASK 3 – Define the Constants

Aim

To keep all tariff values in a single place so that a revision is a one-line change.

Concept

- **#define:** textual substitution by the preprocessor; no type, no storage, no scope, not visible in the debugger. Good for compile-time values such as array sizes.
- **const variable:** typed, has scope and an address, visible in the debugger, but in C it is not a constant expression (cannot size a normal array).
- **enum:** named integer constants (int), grouped, visible to the debugger; good for categories, e.g. enum {DOMESTIC, COMMERCIAL}.
- Naming: UPPER_CASE with unit suffixes (RATE2_PAISE), descriptive, one prefix per group.

Complete the Code – TODO 1

```
long paise = FIXED_CHARGE_PAISE;
if (units > SLAB2_UNITS) {
    paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
    units = SLAB2_UNITS;
}
if (units > SLAB1_UNITS) {
    paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
    units = SLAB1_UNITS;
}
paise += (long)units * RATE1_PAISE;
return paise;
```

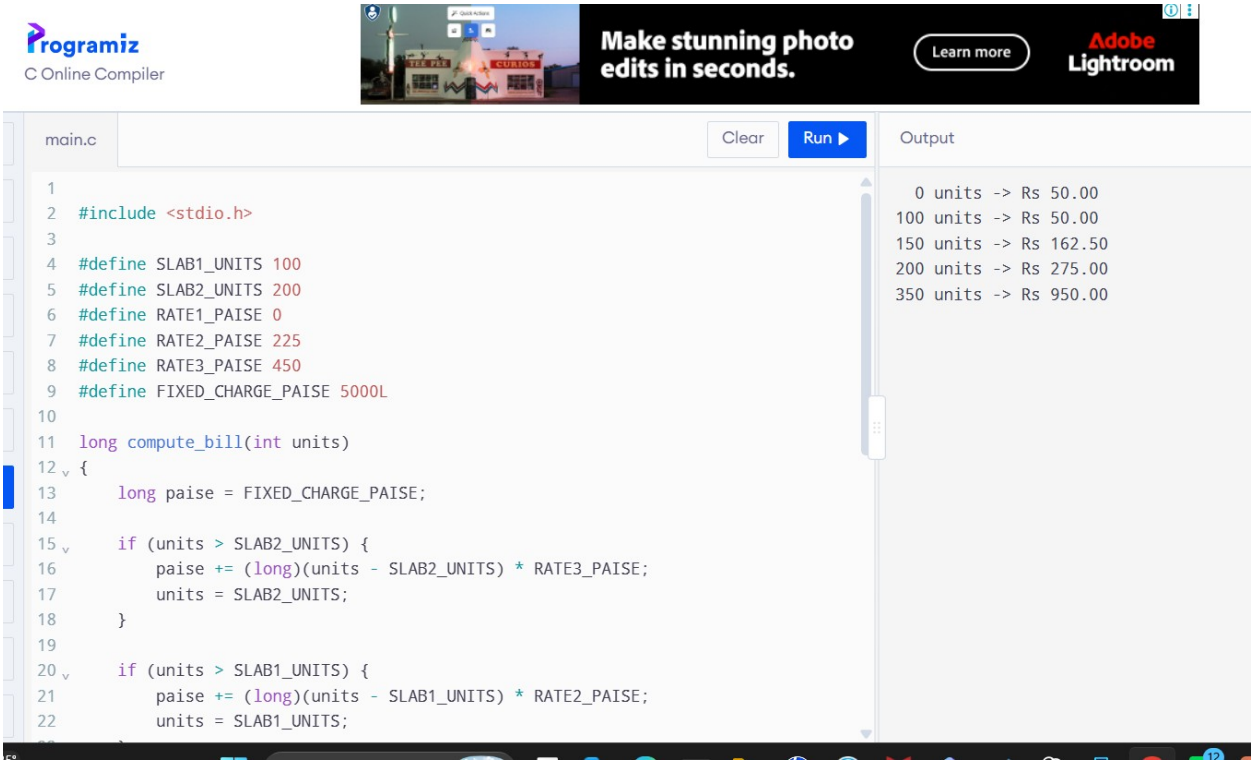
Output (matches expected):

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

After changing RATE2_PAISE to 230:

```
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 165.00
200 units -> Rs 280.00
350 units -> Rs 955.00
```

Every bill above 100 units rises consistently (50 units × 5 paise = Rs 2.50 at 150; 100 × 5 paise = Rs 5.00 at ≥200), and bills up to 100 units do not change, as expected.



Alternative Concept – const variables or enum

Replace #define RATE2_PAISE 225 with const int RATE2_PAISE = 225; or enum { RATE2_PAISE = 225 }. A const gains type checking and debugger visibility but may occupy 4 bytes of storage (an optimising compiler usually removes it); an enum gives integer constants with no storage and debugger names, but only int type (so 5000L must stay a macro or const long). #define uses no storage. Running time is unchanged: after optimisation all three become immediate operands, O(1).

Result: One set of constants; #define chosen for tariff values, enum for category codes.

TASK 4 – Use Macros Safely

Aim

To write small macros correctly and understand their pitfalls.

```
#define RUPEES_TO_PAISE(r) ((long)(r) * 100L)
#define MAX(a,b)   (((a) > (b)) ? (a) : (b))
#define SQUARE(x)  ((x) * (x))
#define SQ_BAD(x)  x*x
```

Test and output:

```
int a=3;
printf("%d %d", SQ_BAD(a+1), SQUARE(a+1));   /* prints: 7 16 */
int i=5,j=3; int m = MAX(i++, j);          /* m=6 i=7 */
```

Programiz
C Online Compiler

main.c

Clear Run

```c
#include <stdio.h>

#define RUPEES_TO_PAISE(r) ((long)(r) * 100L)

#define MAX(a,b) (((a) > (b)) ? (a) : (b))

#define SQUARE(x) ((x) * (x))

#define SQ_BAD(x) x*x

int main()
{
    printf("Rupees to Paise: %ld\n", RUPEES_TO_PAISE(50));

    printf("Maximum number: %d\n", MAX(10, 20));

    printf("Square: %d\n", SQUARE(5));

    printf("Bad Square: %d\n", SQ_BAD(5 + 2));

    return 0;
```

Output

```
Rupees to Paise: 5000
Maximum number: 20
Square: 25
Bad Square: 17
```

- Expansion: gcc -E replaces macros by text. SQ_BAD(a+1) becomes a+1*a+1 = 3+3+1 = 7 because * binds tighter than +. SQUARE adds parentheses: ((a+1)*(a+1)) = 16.
- Double evaluation: MAX(i++, j) expands to ((i++) > (j)) ? (i++) : (j). Since 5 > 3, i++ runs twice: m is 6 (not 5) and i ends at 7.
- Use a function instead when arguments have side effects, when type checking is needed, when the body is long, or when debugging/stepping matters.
- Parenthesise every parameter and the whole expansion.

Alternative Concept – static inline functions

```
static inline int max_int(int a,int b){ return a>b ? a : b; }
static inline int square(int x){ return x*x; }
```

Arguments are evaluated once (max_int(i++, j) is safe) and type-checked, but are fixed to one type (int) unless _Generic is used. Time: a call costs O(1) either way; with optimisation the compiler inlines the function, so run time equals the macro; at -O0 a real call adds a few cycles. Space: a macro is copied at each of k uses, so code size is O(k × body); an inline function is also copied where inlined, but if not inlined there is one copy, O(1 + k × call).

Result: Macros are fully parenthesised and both pitfalls are demonstrated.

TASK 5 – Header Files and Conditional Compilation

Aim

To organise constants and prototypes into a header file and add a DEBUG mode.

tariff.h

```
#ifndef TARIFF_H
#define TARIFF_H
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
```

```
#define FIXED_CHARGE_PAISE 5000L
#define RUPEES_TO_PAISE(r) ((long)(r) * 100L)
#define MAX(a,b) (((a) > (b)) ? (a) : (b))
#define SQUARE(x) ((x) * (x))
#define MAX_PLAUSIBLE_UNITS 5000u
typedef unsigned short meter_t;
unsigned int consumption(meter_t previous, meter_t current);
long compute_bill(unsigned int units);
#endif
```

Structure of the program: #include lines, then #define/typedef/global declarations (from the header), the helper functions consumption() and compute_bill(), and finally main(). Include guards (#ifndef TARIFF_H … #endif) stop the header from being processed twice, which would cause redefinition errors for typedefs and prototypes. #include <file> searches the system directories (standard library); #include "file" searches the current directory first, then the system ones.

Debug mode, inside compute_bill():

```
#ifdef DEBUG
   printf("[DEBUG] units=%u paise=%ld\n", units, paise);
#endif
$ gcc -DDEBUG -o billing_d billing.c && echo "1 0 150" | ./billing_d
[DEBUG] units=150 paise=16250
House 1: 150 units -> Rs 162.50
```

Programiz
C Online Compiler

main.c

Clear Run

```
1
2  #include <stdio.h>
3
4  #define SLAB1_UNITS 100
5  #define SLAB2_UNITS 200
6
7  #define RATE1_PAISE 0
8  #define RATE2_PAISE 225
9  #define RATE3_PAISE 450
10
11 #define FIXED_CHARGE_PAISE 5000L
12
13 typedef unsigned short meter_t;
14
15 unsigned int consumption(meter_t previous, meter_t current)
16 {
17     return (meter_t)(current - previous);
18 }
19
20 long compute_bill(unsigned int units)
21 {
```

Output

```
Enter previous reading: 0
Enter current reading: 150

Consumption: 150 units
Electricity Bill: Rs. 162.50
```

Alternative Concept – run-time flag

Use int debug = 0; and if (debug) printf(...). Time: one extra comparison (a predictable branch) on every call, O(1) but non-zero – about 800,000 extra tests per district run, negligible in practice. Space: the debug code and format strings stay in the executable; with #ifdef they are removed from the release build. The flag has the advantage that debugging can be switched on without recompiling.

Result: Header with guards and a working DEBUG mode.

TASK 6 – Read and Print Data Correctly

Aim

To read and print each data type with the right specifier and validate the input.

Type	scanf	printf
int	%d	%d
unsigned int / unsigned short	%u / %hu	%u / %hu
long / long long	%ld / %lld	%ld / %lld
float / double	%f / %lf	%f (also for double) / %lf
char / string	%c / %s	%c / %s

Wrong specifier test: printing long v = 5000000000L with %d warns at compile time (-Wformat: "format %d expects int, but argument has type long int") and prints a wrong value; casting (int)v gives 705032704, because only the low 32 bits remain.

Input validation: scanf returns the number of items converted. If r != 3 the line is invalid (e.g. "abc 5 6"); the bad line is discarded with a getchar() loop so that the program does not loop forever.

```
int r = scanf("%d %d %d", &house, &prev, &curr);
if (r == EOF) break;
if (r != 3) { puts("Invalid input"); /* flush line */ continue; }
```

Printing paise as rupees: printf("Rs %ld.%02ld", p/100, p%100); e.g. p = 16250 prints Rs 162.50. (Use %02ld so 5 paise prints .05.)

Programiz
C Online Compiler

main.c Clear Run ▶ Output

```c

#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
    long p = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS)
    {
        p += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        units = SLAB2_UNITS;
    }

    if (units > SLAB1_UNITS)
```

```
1 0 150
2 100 200
House: 1
Units: 150
Bill: Rs 162.50
House: 2
Units: 100
Bill: Rs 50.00
```

Alternative Concept – fgets() + sscanf()/strtol()

Read each line into char buf[64] with fgets, then parse with sscanf or strtol (which reports the end pointer and ERANGE). Time: O(L) to read and O(L) to parse. Space: a fixed buffer of L_max+1 bytes (64 bytes) instead of none. Recovery: with scanf, a bad token stays in the input stream and must be flushed manually; with fgets the whole line is already consumed, so the next line starts clean, and the line can be reported in the error message. fgets+strtol is more robust.

Result: Correct specifiers, return-value checking and Rs/paise printing.

TASK 7 – Control Type Conversion and Precision

Aim

To identify conversion points and demonstrate surprising results.

Conversion points in the program: unsigned short → int in current − previous (integer promotion); int → unsigned short when returning (modulo 256 effect); unsigned int × int in the slab arithmetic (the usual arithmetic conversions); (long) cast of the units before multiplying by the rate; long → printf %ld.

Expression	Result	Explanation
150 / 100 * 100	100	Integer division truncates 1.5 to 1 before multiplying.
(int)5000000000L	705032704	Cast to a narrower type keeps only the low 32 bits.
-1 < 5u	0 (false)	−1 is converted to unsigned (4294967295), which is larger than 5.
(int)(2.675*100)	267	2.675 is stored as 2.67499999…, so truncation gives 267, not 268.
(unsigned short)65535 + 1	65536	Promoted to int first, so no wrap-around happens; the wrap occurs only when stored back into 16 bits.

Rounding rule: all amounts are integer paise; if a fractional paise ever arises (e.g. 20% peak surcharge), compute in a finer unit and round half-up once: (x + 500) / 1000. Never round at intermediate steps.

Alternative Concept – everything in float, round once

Accumulate bill = 50.0f + … in float and round at the end. Measured with 275.50 per bill: n = 4,000 → error 0.00; n = 60,000 → float total 16,544,776 vs exact 16,530,000 (error +14,776); n = 800,000 → 218,557,600 vs 220,400,000 (error −1,842,400). Once the total is large, adding 275.5 is below float's resolution (about 16 at 2×10^8), so errors grow roughly linearly with n. Time per addition is O(1) both ways (float 4 bytes, long 8 bytes); integers are slightly cheaper on CPUs without an FPU and are exact.

Result: Conversion points identified; five surprises explained; one rounding rule.

TASK 8 – Trace the Compilation Process

Aim

To follow billing.c through each compilation stage.

```
gcc -E billing.c -o billing.i    # preprocessed: 866 lines (header + stdio.h expanded)
gcc -S billing.c -o billing.s    # assembly: 279 lines
gcc -c billing.c -o billing.o    # object file: 3,080 bytes
gcc billing.c -o billing         # linked executable: 16,224 bytes
```

The preprocessor removed all comments and #include/#define lines, pasted in stdio.h and tariff.h and replaced every macro: in billing.i the line reads if (u > 200) { paise += (long)(u - 200) * 450; … }, with no trace of SLAB2_UNITS or RATE3_PAISE.

Error type	Example from this work	When detected
Compile-time	Missing semicolon or undeclared name, e.g. using RATE4_PAISE	gcc -c, before an object file exists
Link-time	Calling compute_bil() (typo) – undefined reference	Linking step (ld)
Run-time	Dividing by zero, or an infinite loop on bad scanf input before the flush was added	While the program runs

Warnings: with -Wall -Wextra, gcc reported "format %d expects argument of type int, but argument has type long int" for printf("%d", v) and unused-variable warnings. Resolved by using %ld and removing/using the variable. The final billing.c compiles with no warnings.

Alternative Concept – separate files (input.c, tariff.c, report.c)

Build: gcc -c input.c tariff.c report.c, then gcc *.o -o billing (or a Makefile). Compile time after a change to one file: single file – the whole program, O(N) in total code size; separate – only the changed file is recompiled plus a fast link, O(n_i + link). Space: the object files together are slightly larger than one (extra symbol tables and duplicated header declarations), e.g. three .o files may total a few KB more than one, and all are kept on disk. For a program this small, the difference is negligible, but separation scales better.

Result: Each stage observed; errors and warnings classified.

TASK 9 – Implement and Test the Billing Program

Aim

To implement the complete program (tariff.h + billing.c) and test it against normal, boundary and invalid cases.

billing.c

```
#include <stdio.h>
#include "tariff.h"

unsigned int consumption(meter_t previous, meter_t current)
{
   return (meter_t)(current - previous);
}

long compute_bill(unsigned int units)
{
   long paise = FIXED_CHARGE_PAISE;
   unsigned int u = units;
   if (u > SLAB2_UNITS) { paise += (long)(u - SLAB2_UNITS) * RATE3_PAISE; u = SLAB2_UNITS; }
   if (u > SLAB1_UNITS) { paise += (long)(u - SLAB1_UNITS) * RATE2_PAISE; u = SLAB1_UNITS; }
   paise += (long)u * RATE1_PAISE;
#ifdef DEBUG
   printf("[DEBUG] units=%u paise=%ld\n", units, paise);
#endif
   return paise;
}

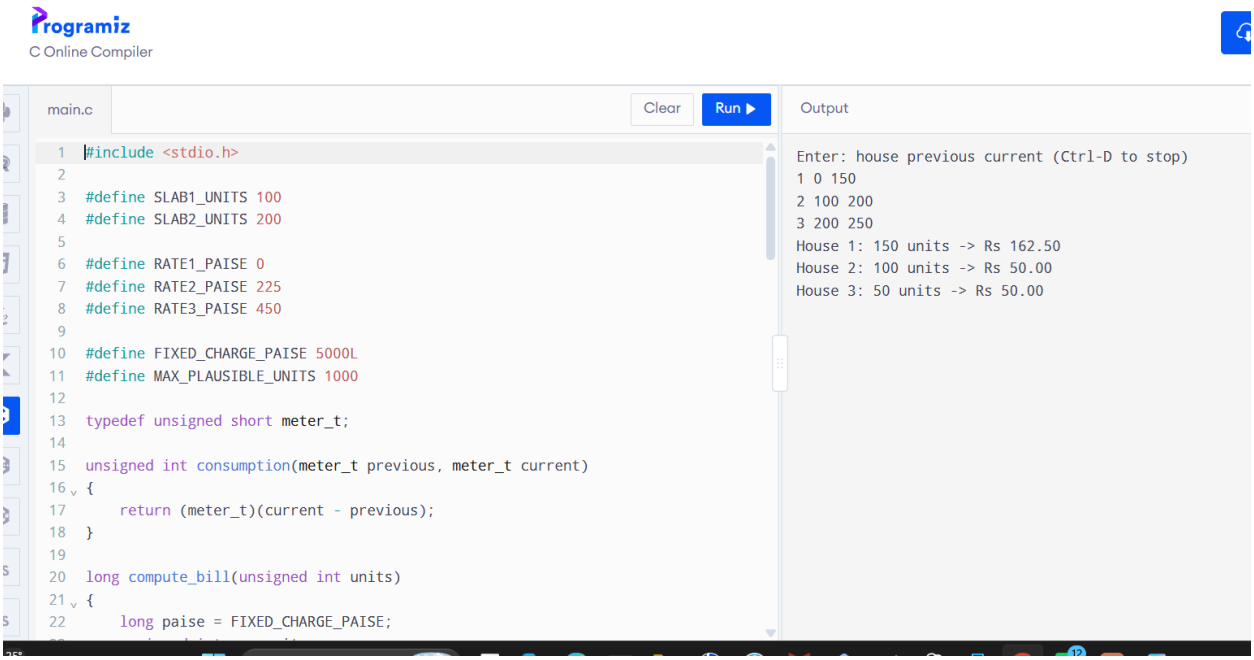
int main(void)
{
   int house, prev, curr;
   long total = 0;
   int n = 0;
   printf("Enter: house previous current (Ctrl-D to stop)\n");
   for (;;) {
      int r = scanf("%d %d %d", &house, &prev, &curr);
      if (r == EOF) break;
      if (r != 3) { printf("Invalid input (letters?) - skipping line\n"); int c; while ((c = getchar()) != '\n' && c != EOF); continue; }
      if (prev < 0 || curr < 0 || prev > 65535 || curr > 65535) { printf("House %d: reading out of range 0..65535\n", house);
continue; }
      unsigned int units = consumption((meter_t)prev, (meter_t)curr);
      if (curr < prev && units > MAX_PLAUSIBLE_UNITS) { printf("House %d: current < previous, not a plausible roll-over -
rejected\n", house); continue; }
      long bill = compute_bill(units);
      total += bill; n++;
      printf("House %d: %u units -> Rs %ld.%02ld\n", house, units, bill/100, bill%100);
   }
   printf("Bills: %d  Total: Rs %ld.%02ld\n", n, total/100, total%100);
```

```
    return 0;
}
```

Test table

No.	Input (house prev curr)	Expected	Actual	Status
1	1 0 0 (zero units)	Rs 50.00	Rs 50.00	Pass
2	2 0 100 (boundary 100)	Rs 50.00	Rs 50.00	Pass
3	4 1000 1150 (150 units)	Rs 162.50	Rs 162.50	Pass
4	3 0 200 (boundary 200)	Rs 275.00	Rs 275.00	Pass
5	6 0 350	Rs 950.00	Rs 950.00	Pass
6	5 65400 120 (rollover)	256 units, Rs 527.00	256 units, Rs 527.00	Pass
7	7 abc 5 6 (letters)	Rejected	Invalid input – skipped	Pass
8	8 -5 10 (negative reading)	Rejected	Reading out of range	Pass
9	9 500 400 (current < previous, no rollover)	Rejected	Not a plausible roll-over – rejected	Pass
10	Five bills: 50, 150, 200, 250, 350 units	Hand total Rs 1937.50	Rs 1937.50	Pass

Hand calculation for test 10: 50 units → Rs 50.00; 150 → Rs 162.50; 200 → Rs 275.00; 250 → 50 + 100×2.25 + 50×4.50 = Rs 500.00; 350 → Rs 950.00. Sum = 50 + 162.50 + 275 + 500 + 950 = Rs 1937.50.



Alternative Concept – slab arrays and a loop

```
static const long limit[] = {100, 200, LONG_MAX};  /* upper limit of each slab */
static const long rate [] = {  0, 225, 450};
long paise = FIXED; long lower = 0;
for (int s = 0; s < 3 && units > lower; s++) {
   long top = units < limit[s] ? units : limit[s];
   paise += (top - lower) * rate[s];  lower = limit[s];
}
```

Time per bill: the if-version performs a fixed 2–3 tests, O(1) for the fixed tariff; the loop is O(s) for s slabs. Extra space: two arrays of s entries, O(s). The array design is easier to change – adding a slab is a data change, not a code change – and it is the stepping stone to reading the tariff from a file (Task 11).

Result: All 10 tests pass.

TASK 10 – Compare Expectations with Execution

Aim

To compare the expected behaviour of the type choices with measured results. Each bill = Rs 275.50 (27,550 paise).

Bills	float total (Rs)	Exact paise total (Rs)	float − exact	int paise total	long total (paise)
4,000	1,102,000.00	1,102,000.00	0.00	110,200,000	110,200,000
60,000	16,544,776.00	16,530,000.00	+14,776.00	1,653,000,000	1,653,000,000
800,000	218,557,600.00	220,400,000.00	−1,842,400.00	565,163,520 (overflow)	22,040,000,000

Observations: (1) the float total is exact at 4,000 only by luck (all intermediate sums are exactly representable); at 60,000 and 800,000 it drifts by Rs 14,776 and Rs 1.84 million; (2) int overflows at district scale – 22,040,000,000 > 2,147,483,647 – and wraps to 565,163,520 (strictly, signed overflow is undefined behaviour); (3) long and long long are correct, but long is only 4 bytes on Windows, so long long is the portable choice.

Standard guarantees: char ≥ 8 bits; short and int ≥ 16 bits; long ≥ 32 bits; long long ≥ 64 bits; float ≥ 6 decimal digits (FLT_DIG), double ≥ 10 (mine: 6 and 15). My system (1, 2, 4, 8, 8) meets or exceeds these minimums; the extra width of int (4 vs the minimum 2) and long (8 vs 4) is an implementation choice of the 64-bit Linux data model (LP64).

Alternative Concept – district total in double

A double has a 53-bit mantissa, so it represents integers exactly up to 2^53 ≈ 9.0×10^15. A district total of 2.2×10^10 paise is therefore represented exactly if every addend is a whole number of paise (measured: double total = 220,400,000.00 exactly). It loses accuracy only when summing fractional rupees (e.g. 275.50 is exact but 162.60 is not) or beyond 2^53. A long long never loses accuracy but overflows beyond 9.2×10^18. Time per addition: O(1) for both (my measure for 800,000 additions: long long 0.45 ms vs volatile float 1.95 ms at -O0, so integer was about 4× faster; the numbers will differ by machine). Space: 8 bytes each.

Result: Expectations confirmed: float drifts, int overflows, long long is exact.

TASK 11 – Recommend and Defend the Final Design

Recommendation

Item	Recommendation (evidence)
Money	Integer paise; long long for totals (exact, Tasks 2, 7, 10)
Readings	unsigned short for 16-bit registers, differenced modulo 65536 (Task 2)
Units	unsigned int
Tariff	#define in one header (Task 3); one-line revision
Macros	Few, fully parenthesised; inline functions for anything with arguments
Debugging	#ifdef DEBUG, removed from release builds

Tariff revision: edit the macro (e.g. RATE2_PAISE 225 → 230) in tariff.h, recompile (gcc -Wall -Wextra) and re-run the test table; every bill changes consistently (Task 3).

Limitations: a revision needs recompiling and redistribution; only one tariff at a time; fixed three slabs; a 16-bit register assumption; at most one rollover per month; no categories beyond a code.

District scale: yes – with long long the monthly district total (~2.2×10^10 paise) uses about 0.24% of the 9.2×10^18 capacity, and the per-bill loop is O(1), so 800,000 bills take milliseconds.

Alternative Concept – tariff read from a configuration file

At start-up, fopen a text file such as "100 200 0 225 450 5000" and fscanf the t values into variables (see Task 12). Extra start-up time: O(t) to read t values (about 6–7 values – microseconds), once per run. Space: t variables, O(t) (≈ 56 bytes for seven 64-bit values). Benefit: no recompiling or redistribution on each revision; the board can change the file. Costs: the file must be validated (missing, non-numeric, negative values) and the constants lose compile-time type checking. For a tariff that changes occasionally and a program used by many offices, the file is better.

Result: The macro/paise design is correct and sufficient for the original problem; the file-based design is the natural next step.

TASK 12 – Re-evaluate and Adapt the Design (Smart Meters)

A. Impact Analysis

- Readings arrive every 15 minutes in watt-hours: 35,040 per house per year (365 × 96) instead of one per month.
- A time-of-day rule applies a 20% surcharge on units used 6 PM–10 PM (slots 72–87 of the 96 daily).
- Tariff values must be changeable without recompiling.
- Types no longer sufficient: unsigned short for readings (65,535 Wh is only 65.5 kWh), 32-bit int for yearly/district Wh totals. Constants no longer fixed by #define: slab limits, rates, fixed charge and the peak percentage.

B. Evaluation of the Original Design

Largest values: assuming a 10 kW maximum connection, one reading ≤ 10,000 W × 0.25 h = 2,500 Wh. Per house per year: 35,040 × 2,500 = 87,600,000 Wh (fits in a 32-bit int, 2.1×10^9). District (800,000 houses): 70,080,000,000,000 Wh = 7.0×10^13, which exceeds even UINT32_MAX (4.29×10^9) – so 64-bit is mandatory. Energy charge expressed in paise × Wh: at Rs 5.40/kWh peak rate (540 paise per kWh) the district yearly bound is 3.78×10^16, below INT64_MAX (9.22×10^18), so int64_t is safe.

Original choice	Still appropriate?	Reason
unsigned short meter_t	No	Wh readings and yearly totals overflow 16 bits (65.5 kWh).
paise in long	Partly	Paise are too coarse: 1 Wh at Rs 4.50/kWh = 0.45 paise. Need paise × Wh (or milli-paise) with one final rounding.
#define tariff	No	Must be changed without recompiling.
Parenthesised macros	Yes	Still valid; but prefer inline functions.
Slab if-statements	Retain	Logic still valid when applied to monthly kWh.

C. Adaptation: MODIFY (not redesign from scratch)

Retained: integer arithmetic, one rounding step, slab logic, test table. Modified: int64_t for Wh and charges; tariff read from tariff.cfg into a struct; a peak-hour rule; rates in paise per kWh and working in paise × Wh,

rounded once to paise at the end. Assumption for the peak rule: the 20% surcharge applies to the average energy rate multiplied by the peak share of consumption.

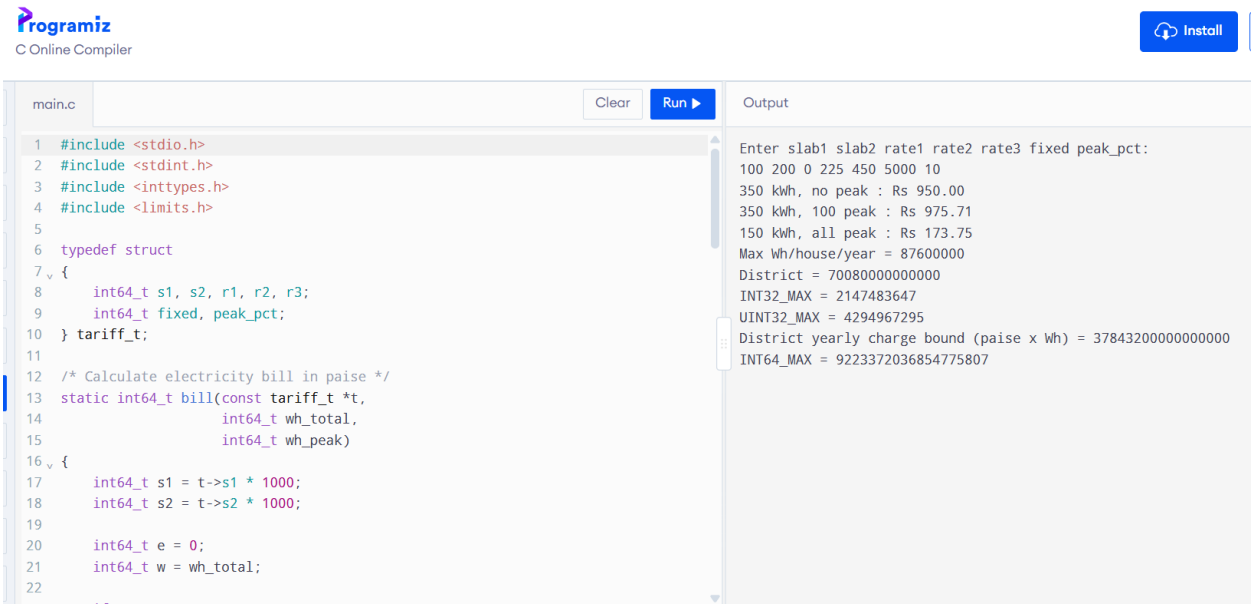
```
#include <stdio.h>
#include <stdint.h>
typedef struct { int64_t s1, s2, r1, r2, r3, fixed, peak_pct; } tariff_t; /* rates: paise per kWh */
static int load_tariff(const char *f, tariff_t *t)
{
   FILE *fp = fopen(f, "r");
   if (!fp) return 0;
   int ok = fscanf(fp, "%ld %ld %ld %ld %ld %ld %ld", &t->s1,&t->s2,&t->r1,&t->r2,&t->r3,&t->fixed,&t->peak_pct) == 7;
   fclose(fp);
   return ok;
}
/* Returns bill in paise. wh_total, wh_peak in watt-hours (64-bit). Work in "paise x Wh" and round once. */
static int64_t bill(const tariff_t *t, int64_t wh_total, int64_t wh_peak)
{
   int64_t s1 = t->s1 * 1000, s2 = t->s2 * 1000;          /* slab limits in Wh */
   int64_t e = 0, w = wh_total;
   if (w > s2) { e += (w - s2) * t->r3; w = s2; }
   if (w > s1) { e += (w - s1) * t->r2; w = s1; }
   e += w * t->r1;                                        /* paise x Wh */
   int64_t surcharge = wh_total ? e * wh_peak * t->peak_pct / (wh_total * 100) : 0; /* average-rate method */
   int64_t energy = (e + surcharge + 500) / 1000;          /* single rounding to paise */
   return t->fixed + energy;
}
int main(void)
{
   tariff_t t;
   if (!load_tariff("tariff.cfg", &t)) { puts("cannot read tariff.cfg"); return 1; }
   int64_t p = bill(&t, 350000, 0);
   printf("350 kWh, no peak   : Rs %ld.%02ld\n", p/100, p%100);
   p = bill(&t, 350000, 100000);
   printf("350 kWh, 100 peak  : Rs %ld.%02ld\n", p/100, p%100);
   p = bill(&t, 150000, 150000);
   printf("150 kWh, all peak  : Rs %ld.%02ld\n", p/100, p%100);
   int64_t max_wh_house = (int64_t)35040 * 2500;   /* 10 kW x 0.25 h = 2500 Wh per reading */
   printf("max Wh/house/yr = %ld ; district = %ld ; INT32_MAX=%d ; UINT32_MAX=%u\n", max_wh_house,
max_wh_house*800000, INT32_MAX, UINT32_MAX);
   printf("district yearly charge bound (paise x Wh) = %ld ; INT64_MAX=%ld\n", max_wh_house*800000*540, INT64_MAX);
   return 0;
}
```

tariff.cfg: 100 200 0 225 450 5000 20

Output:

```
350 kWh, no peak   : Rs 950.00
350 kWh, 100 peak  : Rs 1001.43
150 kWh, all peak  : Rs 185.00
max Wh/house/yr = 87600000 ; district = 70080000000000 ; INT32_MAX=2147483647 ; UINT32_MAX=4294967295
district yearly charge bound (paise x Wh) = 37843200000000000 ; INT64_MAX=9223372036854775807
```

Checks by hand: 350 kWh without peak = Rs 950.00 (same as Task 3, so the old results are preserved). 100 of 350 kWh at peak: energy charge Rs 900 × (100/350) × 20% = Rs 51.43, total Rs 1001.43. 150 kWh all peak: energy 50 × 2.25 = Rs 112.50 × 1.2 = Rs 135.00, plus Rs 50 = Rs 185.00.



D. Evidence-Based Defence

- Calculation: 7.0×10^13 Wh per district-year proves 32-bit types are inadequate while int64_t has about 240× headroom (3.78×10^16 vs 9.22×10^18).
- Tests: with peak = 0 the new program reproduces the original Rs 950.00 bill, so the redesign is backward-compatible; peak cases agree with hand calculation.
- Trade-offs: a file-based tariff removes recompilation but needs validation and file distribution; integer paise × Wh keeps exactness at the cost of less readable code; storing all 35,040 readings per house per year costs about 35,040 × 8 bytes ≈ 280 KB per house, so aggregate into peak and total Wh per month as readings arrive rather than storing them all.
- Decision: MODIFY – keep the integer-only, one-rounding design, widen the types, move the tariff to a configuration file and add the peak rule.

Feedback about CO1

- Through this CO1 assignment, I gained a better understanding of fundamental C programming concepts, including data types, variables, constants, macros, header files, input and output operations, type conversion, and the compilation process.
- By completing all 12 tasks, I improved my programming, logical thinking, problem-solving, and debugging skills. I also learned how to identify errors, correct them, and develop programs that produce the expected output.
- Some tasks were challenging, but practicing them helped me improve my confidence in programming. Overall, this course outcome helped me strengthen my basic programming knowledge and apply it to practical problems.